

Hepsiba Devara

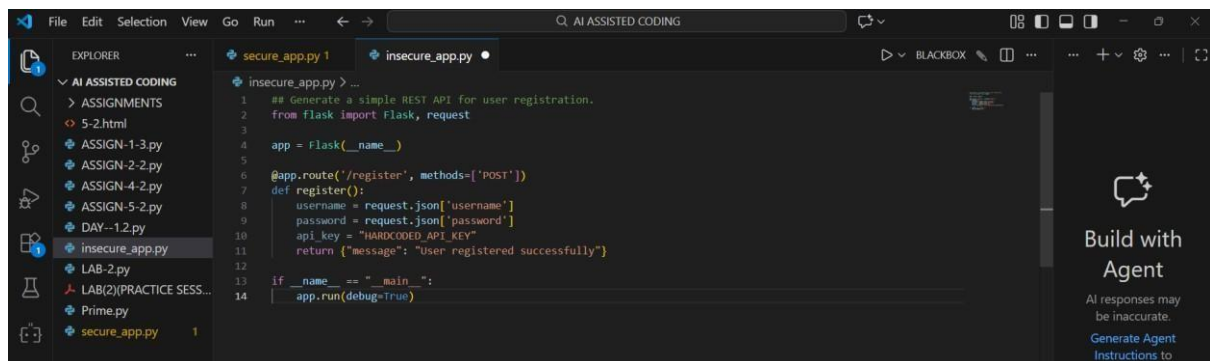
2403a51l25

B-51

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description – 1: Secure API Usage

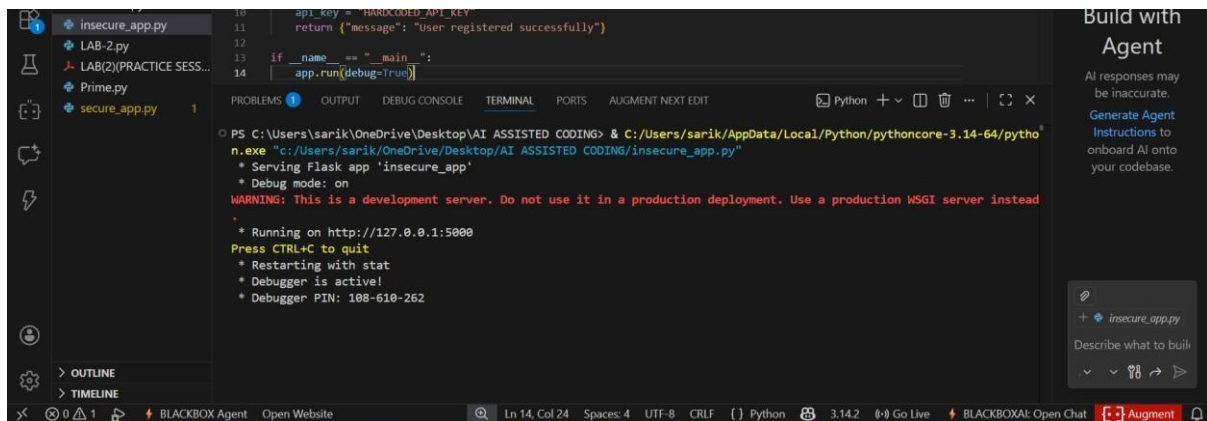
Prompt: Generate a simple REST API for user registration.



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left lists files under 'AI ASSISTED CODING', including 'insecure_app.py'. The main editor displays the code for 'insecure_app.py', which is a simple REST API for user registration using Flask. The code includes a POST endpoint at '/register' that takes a JSON body with 'username' and 'password' fields and returns a success message. The terminal panel at the bottom shows the command to run the application: 'python insecure_app.py', and the output indicates the server is running on http://127.0.0.1:5000.

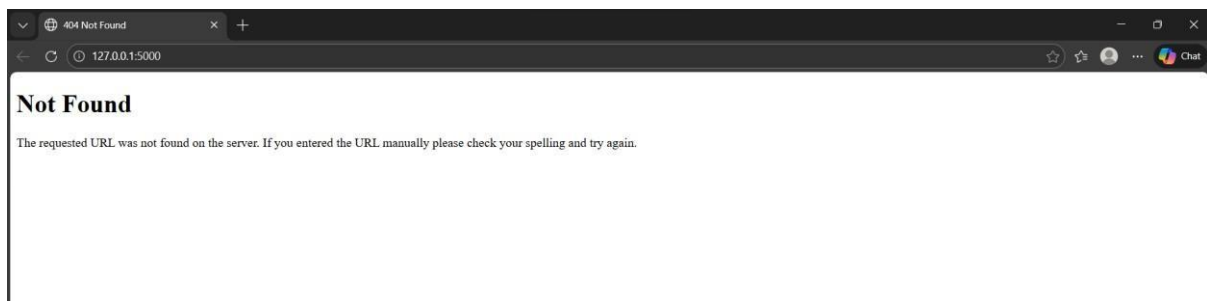
```
1 # Generate a simple REST API for user registration.
2 from flask import Flask, request
3
4 app = Flask(__name__)
5
6 @app.route('/register', methods=['POST'])
7 def register():
8     username = request.json['username']
9     password = request.json['password']
10     api_key = "HARDCODED_API_KEY"
11     return {"message": "User registered successfully"}
12
13 if __name__ == "__main__":
14     app.run(debug=True)
```

OUTPUT:



The screenshot shows the Visual Studio Code interface with the terminal panel open. The terminal displays the output of the command 'python insecure_app.py'. The output shows the server starting on http://127.0.0.1:5000 and serving the Flask app 'insecure_app'. The terminal also shows a warning message: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead'. The status bar at the bottom indicates the file is 'insecure_app.py' and the Python version is 3.14.2.

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/insecure_app.py"
* Serving Flask app 'insecure_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 108-618-262
```



```

9     password = request.json['password']
10    api_key = "HARDCODED_API_KEY"
11    return {"message": "User registered successfully"}
12
13 if __name__ == "__main__":
14     app.run(debug=True)

```

```

PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python
n.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/insecure_app.py"

* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 188-610-262
127.0.0.1 - - [20/Jan/2026 21:46:17] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [20/Jan/2026 21:46:17] "GET /favicon.ico HTTP/1.1" 404 -

```

Explanation: You got 404 error because your Flask app does not have a home (/) route, so the browser cannot find that page.

Identified Security Flaws:

1. API key is **hardcoded**, exposing sensitive credentials
2. No authentication or authorization mechanism
3. No input validation (password strength, missing fields)
4. Password stored/used in **plain text**
5. No token-based access control

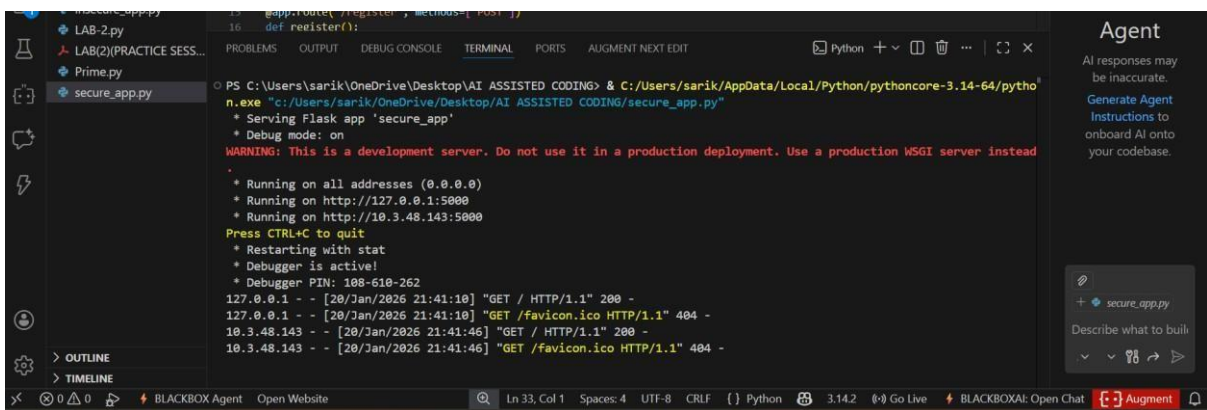
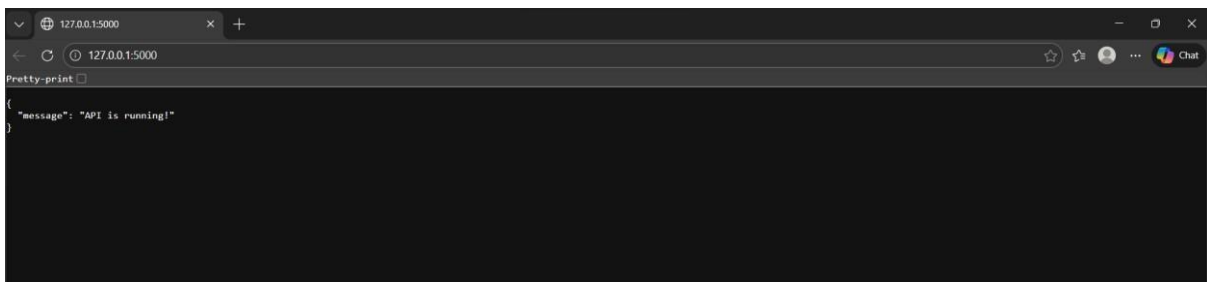
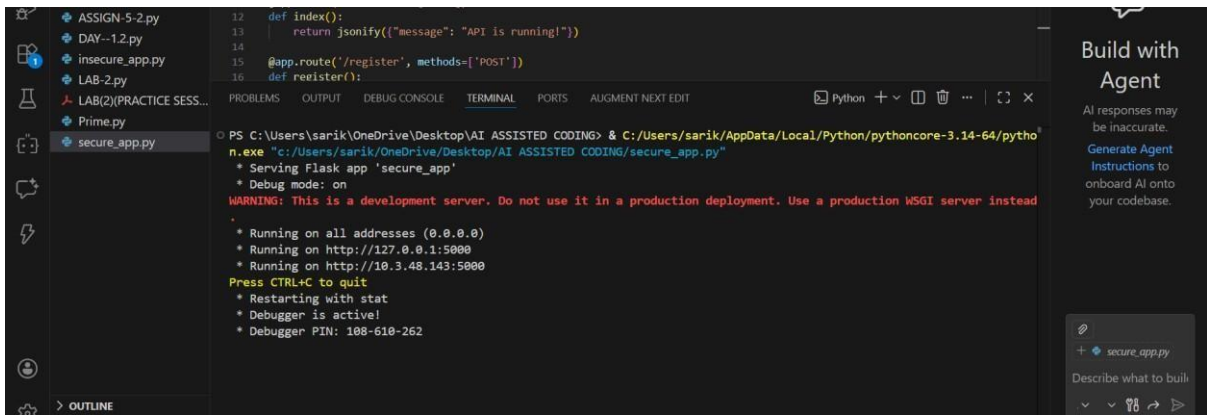
Corrected Secure Version (Token-Based Authentication):

```

1  # Secure API (Corrected - Token-Based Authentication)
2  > from flask import Flask, request, jsonify
3
4  app = Flask(__name__)
5  app.config['SECRET_KEY'] = os.getenv("SECRET_KEY", "mysecretkey")
6
7  @app.route('/', methods=['GET'])
8  def index():
9      return jsonify({"message": "API is running!"})
10
11 @app.route('/register', methods=['POST'])
12 def register():
13     data = request.get_json()
14     if not data or not data.get('username') or not data.get('password'):
15         return jsonify({"error": "Invalid input"}), 400
16     hashed_password = generate_password_hash(data['password'])
17     token = jwt.encode(
18         {
19             'user': data['username'],
20             'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
21         },
22         app.config['SECRET_KEY'],
23         algorithm="HS256"
24     )
25     return jsonify({"token": token})
26
27 if __name__ == "__main__":
28     app.run(debug=True, host="0.0.0.0", port=5000)

```

OUTPUT:



Observations: The initial API code is insecure because it uses a hardcoded API key and does not protect user data. The corrected version improves security by validating inputs, hashing passwords, and using token-based authentication for safer access control.

Task Description – 2: Fair Decision Logic

Prompt: Generate a scholarship eligibility checker based on academic score, family income, and location.

AI-Generated Code:

The screenshot shows a VS Code editor with a file named 'ASSIGN-5-2.py'. The code defines a function `scholarship_eligibility_biased` that takes `score`, `income`, and `location` as arguments. The function's logic is as follows:

```
1  ## Generate a scholarship eligibility checker based on academic score, family income, and location.
2  def scholarship_eligibility_biased(score, income, location):
3      if score > 85 and income < 200000 and location == "urban":
4          return True
5      return False
6
7
```

Observations:

1. The logic unfairly favors urban students
2. Rural or semi-urban students are excluded
3. No flexibility or weighted scoring approach

Improved Version:

The screenshot shows the same VS Code editor with the file 'ASSIGN-5-2.py'. The code now defines a function `scholarship_eligibility_fair` that only considers `score` and `income`. It also includes test calls for both the old and new functions.

```
1  /
2
3  def scholarship_eligibility_fair(score, income):
4      if score >= 80 and income <= 300000:
5          return True
6      return False
7
8  print(scholarship_eligibility_biased(90, 150000, "urban"))
9  print(scholarship_eligibility_fair(82, 250000))
10
11
12
13
14
```

OUTPUT:

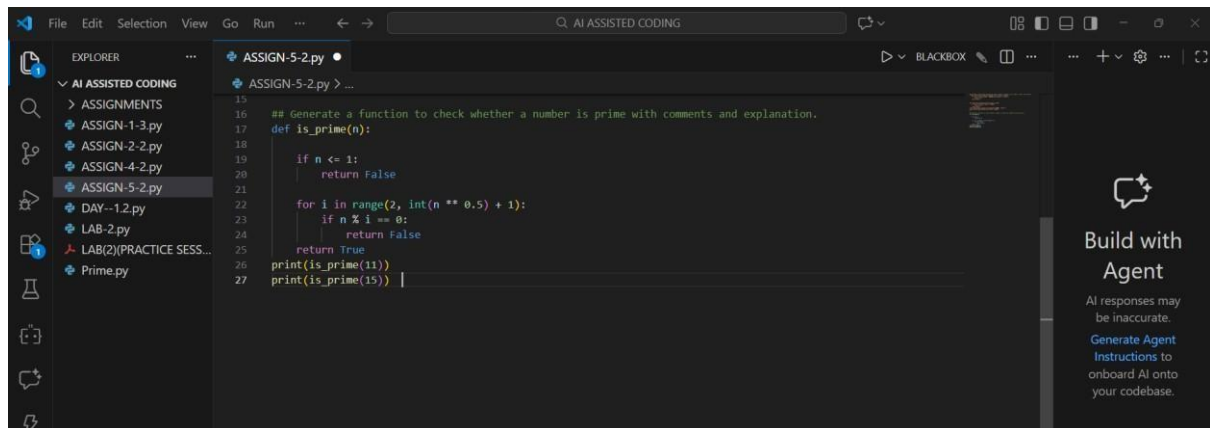
The screenshot shows the VS Code terminal with the command `python n.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-5-2.py"` executed. The output is:

```
True
True
```

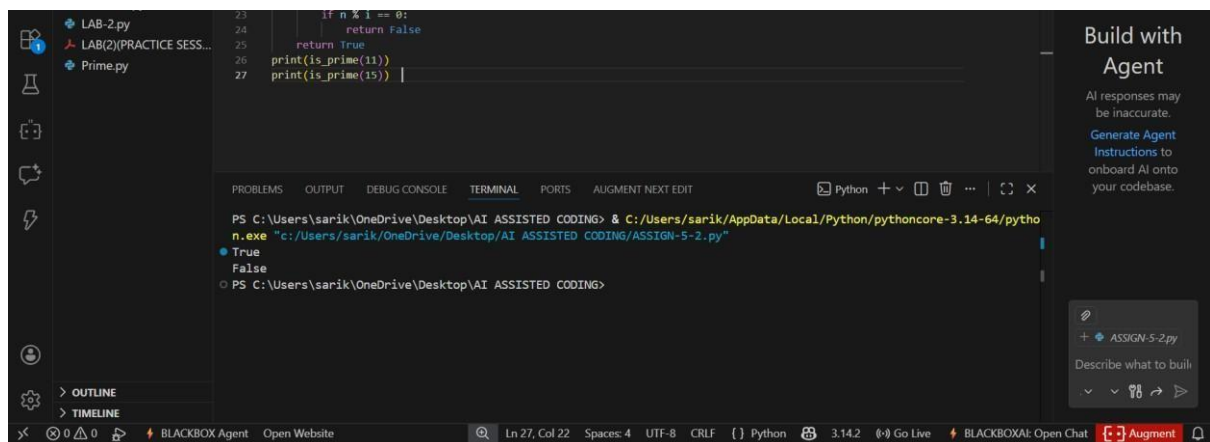
Explanation: The original logic introduces geographic bias by favoring urban students. Location should not be a deciding factor unless justified by policy. A fair system focuses on merit and economic need. Weighted or threshold-based criteria help ensure equitable access.

Task Description – 3: Explainability

Prompt: Generate a function to check whether a number is prime with comments and explanation.



OUTPUT:

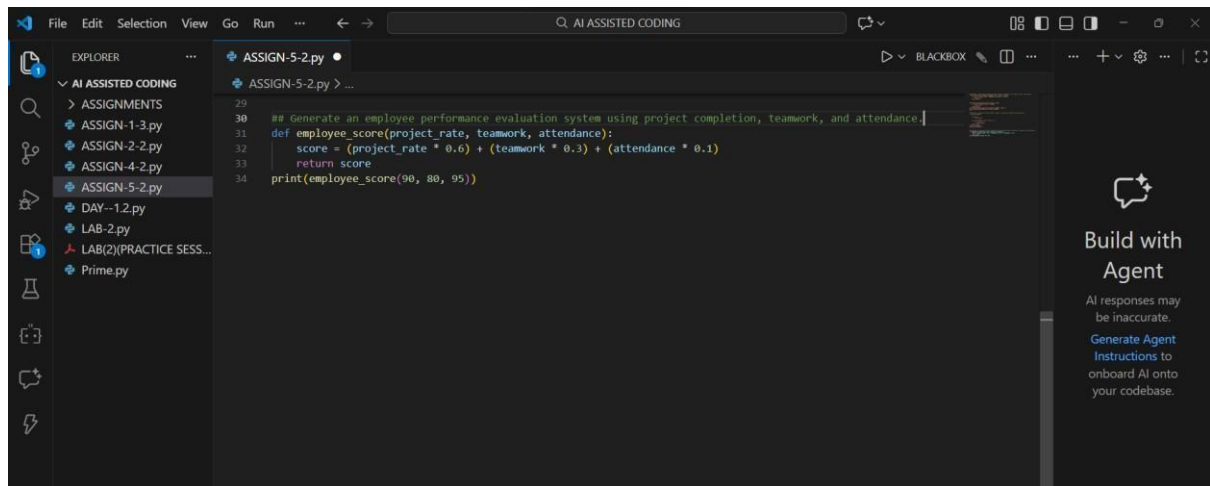


Explanation: The function first checks if the number is greater than 1. It then tests divisibility from 2 up to the square root of the number to reduce computation. If any divisor is found, the number is not prime; otherwise, it is prime.

The explanation is clear, correct, and efficient. Inline comments improve readability and help beginners understand the logic easily.

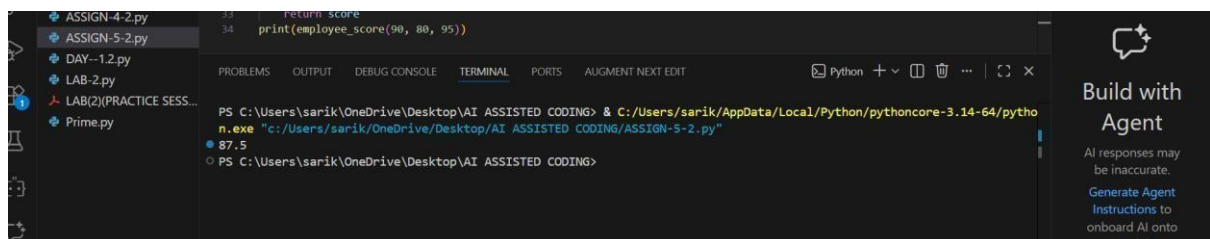
Task Description – 4: Ethical Scoring System

Prompt: Generate an employee performance evaluation system using project completion, teamwork, and attendance.



```
29
30 ## Generate an employee performance evaluation system using project completion, teamwork, and attendance
31 def employee_score(project_rate, teamwork, attendance):
32     score = (project_rate * 0.6) + (teamwork * 0.3) + (attendance * 0.1)
33     return score
34 print(employee_score(90, 80, 95))
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-5-2.py"
87.5
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Observations:

1. Heavy weight on project completion may disadvantage collaborative roles
2. Attendance weighting may penalize employees with health or caregiving needs
3. Teamwork score depends on subjective evaluation

The criteria are reasonable but require transparency and flexibility. Ethical systems should allow contextual review and avoid over-reliance on single metrics.

Task Description – 5: Accessibility and Inclusiveness Prompt:

Generate a user feedback form application.


```
5-2.html
File Edit View

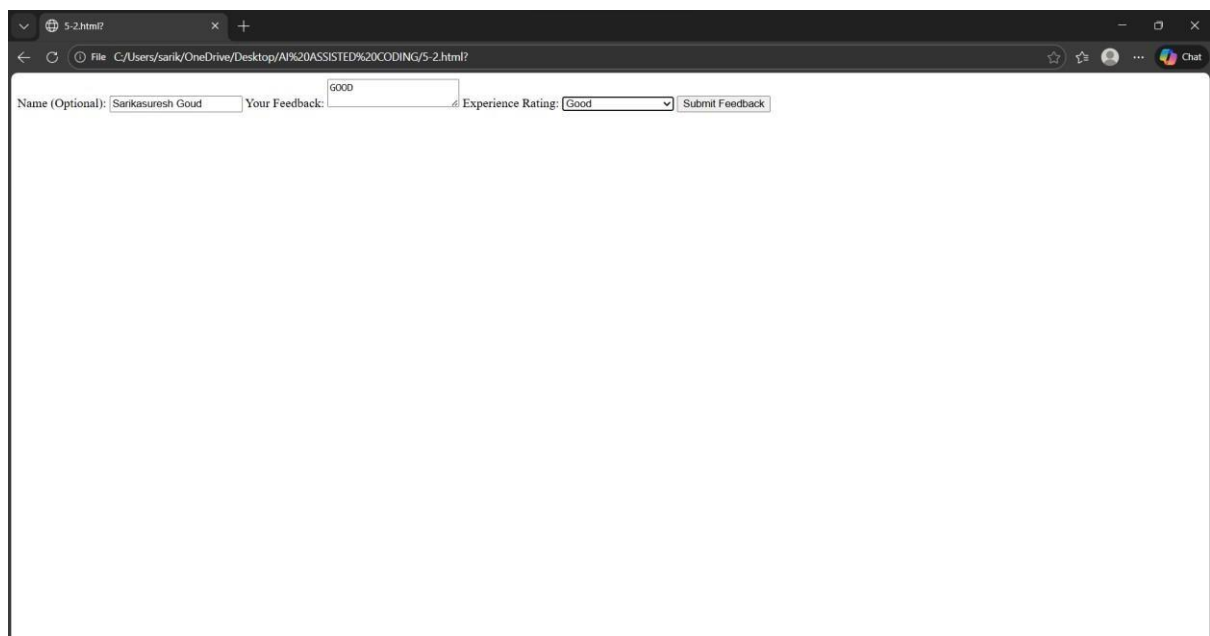
<form aria-label="User Feedback Form">
  <label for="name">Name (Optional):</label>
  <input type="text" id="name" aria-required="false">

  <label for="feedback">Your Feedback:</label>
  <textarea id="feedback" aria-required="true"></textarea>

  <label for="rating">Experience Rating:</label>
  <select id="rating">
    <option>Very Good</option>
    <option>Good</option>
    <option>Neutral</option>
    <option>Needs Improvement</option>
  </select>

  <button type="submit">Submit Feedback</button>
</form>
```

OUTPUT:



The screenshot shows a web browser window with the URL `C:/Users/sarik/OneDrive/Desktop/AI%20ASSISTED%20CODING/5-2.html?` in the address bar. The rendered form is displayed below the browser window. It consists of three input fields: a text field for 'Name (Optional):' containing the text 'Sarikasurosh Goud', a text area for 'Your Feedback:' containing the text 'GOOD', and a dropdown menu for 'Experience Rating:' with 'Good' selected. A 'Submit Feedback' button is positioned to the right of the dropdown menu.`

Observations: The feedback form uses neutral and inclusive language to avoid exclusion of any user group. Accessibility is enhanced through ARIA labels, optional fields, and simple input options for diverse users.